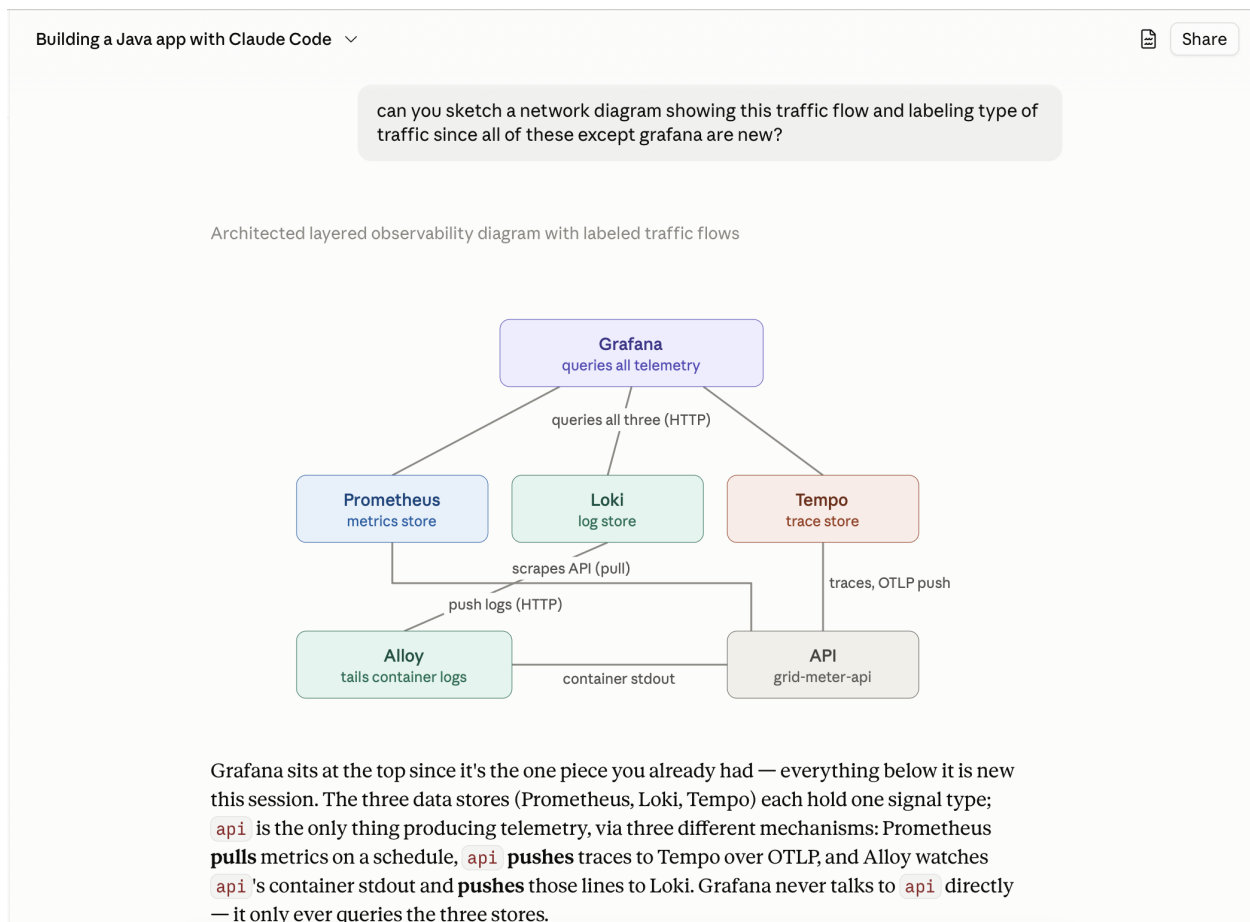


grid-meter-app — traffic flow

Two diagrams: the original observability traffic flow, and the data/messaging tier that has grown up around it since.

1. Observability traffic flow (original, still accurate)

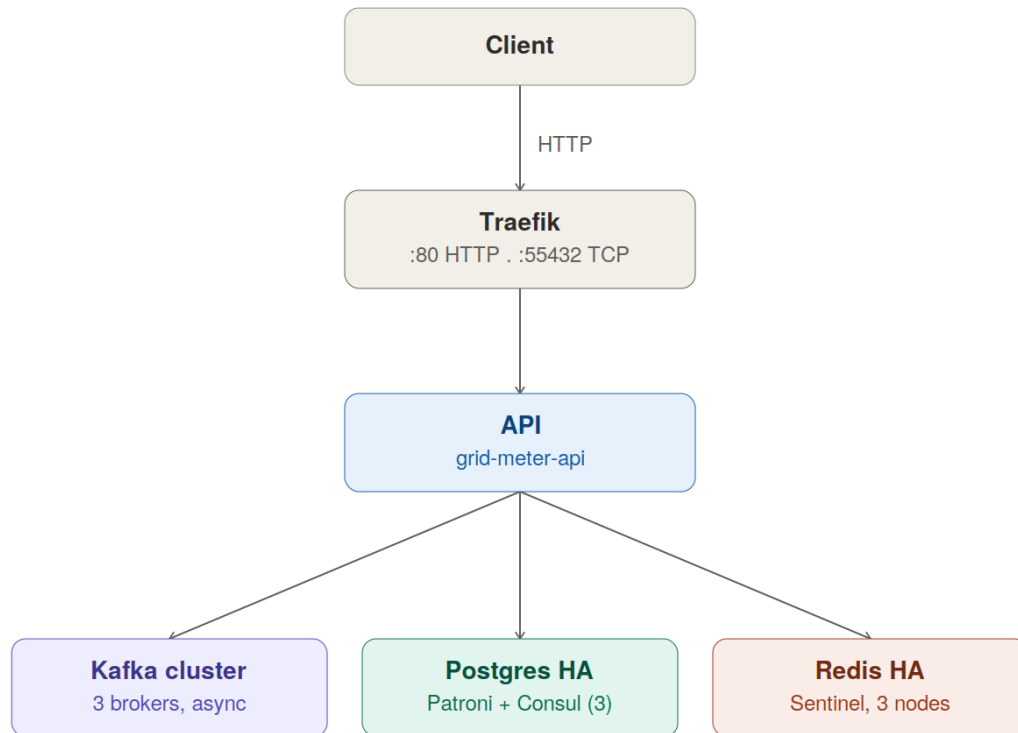
Grafana sits at the top since it's the one piece that already existed — everything below it was new at the time this was drawn. The three data stores (Prometheus, Loki, Tempo) each hold one signal type; the API is the only thing producing telemetry, via three different mechanisms: Prometheus pulls metrics on a schedule, the API pushes traces to Tempo over OTLP, and Alloy watches the API's container stdout and pushes those lines to Loki. Grafana never talks to the API directly — it only ever queries the three stores.



Nothing in the data-tier work described below changed how these five pieces talk to each other.

2. Data and messaging tier (new)

Everything below the API in this diagram is new since the observability flow above was drawn: a 3-broker Kafka cluster, a 3-node Patroni/Consul Postgres HA cluster, and a Sentinel-backed Redis HA cluster, all reached through different mechanisms.



Traffic types

- **Client → Traefik:** HTTP on :80 — the same endpoint as before.
- **Traefik → API:** path-based routing, unchanged.
- **API → Kafka cluster:** producer publishing reading events (acks=all), fully async — the consumer that writes to Postgres/Redis lives inside the same API service, not a separate one.
- **API → Postgres HA:** JDBC, but not a direct connection — it goes back out through Traefik's :55432 TCP endpoint, which routes to whichever of the 3 Patroni nodes is currently primary via Consul Catalog service discovery. Simplified in the diagram as one arrow rather than drawing the loop back through Traefik, since the label captures the real mechanism without cluttering the layout.
- **API → Redis HA:** Lettuce, Sentinel-aware — the client itself asks Sentinel which node is currently master, no proxy involved (a structurally different mechanism from the Postgres path).

Worth remembering

Traefik is drawn once in this diagram but is really doing two unrelated jobs simultaneously — reverse-proxying app HTTP traffic on :80, and TCP-proxying Postgres traffic on :55432 for a completely different client (the API itself, not the browser). Worth keeping in mind if this ever gets walked through in an interview, since it's easy to describe Traefik as "the load balancer" and gloss over that it's also acting as the Postgres HA cluster's client-facing router.